

INFORME TÉCNICO

Detección de velocidad transaccional con Kafka y Apache Beam

Autora: Fátima Barrios Ortega
Programa: Maestría en Inteligencia Artificial - FPUNA
Asignatura: Streaming de datos y sus aplicaciones
Entorno de ejecución validado: Apache Beam DirectRunner
Fecha: 15 de agosto de 2026

Resumen

El proyecto implementa un pipeline end-to-end para detectar velocidad transaccional en un contexto de prevención de lavado de dinero y fraude. Un productor sintético publica eventos versionados en Kafka; Apache Beam valida, asigna tiempo de evento, aplica ventanas deslizantes, deduplica, agrega y genera alertas con claves idempotentes; Kafka conserva tanto los resultados como los registros inválidos.

Resultado de validación: ocho eventos consumidos, tres alertas válidas, cero eventos inválidos, LAG final igual a cero y código de salida del pipeline igual a cero.

La implementación final usa DirectRunner para una validación local reproducible. La política temporal completa se verifica mediante TestStream; la ejecución acotada de ocho registros utiliza un modo específico que desactiva únicamente el disparo temprano dependiente del reloj de procesamiento.

1. Caso de uso y valor operativo

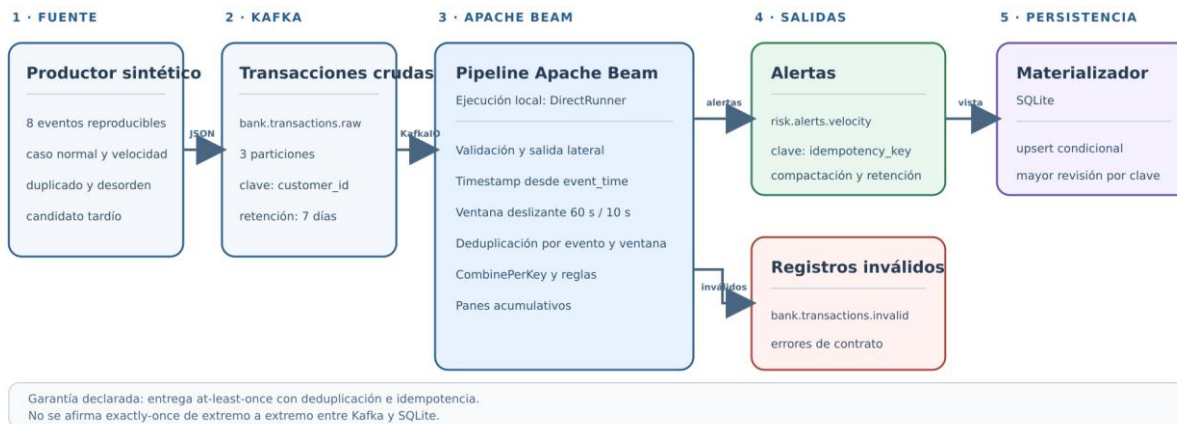
Una regla aislada por transacción puede no detectar una secuencia fragmentada. El pipeline observa la actividad de cada cliente en intervalos solapados y produce una señal cuando el conteo o el monto acumulado supera el umbral. Los usuarios del resultado son analistas y procesos de monitoreo que necesitan priorizar revisiones y explicar por qué se disparó una alerta.

Condición	Umbral	Decisión habilitada
Conteo	4 o más transacciones	Priorizar concentración operativa
Monto	10.000.000 PYG o más	Priorizar exposición acumulada

Fuente: Regla implementada en `src/pipeline/alerts.py`.

La alerta no constituye una decisión final de sospecha. Es una señal auditable que incluye cliente, ventana, conteo, monto, umbrales y condiciones disparadas.

2. Arquitectura y flujo end-to-end



Fuente: Elaboración propia a partir de la implementación final.

- Fuente:** genera ocho eventos reproducibles con casos normal, duplicado, fuera de orden y candidato tardío.
- Kafka:** conserva el input crudo, desacopla fuente y procesamiento y permite replay por offsets.
- Beam:** expresa validación, tiempo de evento, ventanas, estado, agregación y reglas.
- Salida:** publica alertas con clave estable y errores de contrato en un tópico separado.
- Materializador:** persiste en SQLite la revisión numéricamente mayor por clave idempotente.

3. Contrato de eventos y Kafka

3.1 Contrato versionado

Campo	Tipo	Función
<code>schema_version</code>	string	Evolución explícita; versión actual 1.0
<code>event_id</code>	string	Identificador estable para deduplicación
<code>customer_id</code>	string	Clave de negocio, partición y agregación
<code>event_time</code>	ISO 8601 UTC	Tiempo del hecho en el dominio
<code>payload.amount</code>	integer	Monto positivo en PYG
<code>payload.transaction_type</code>	enum	Tipo de transacción permitido
<code>payload.channel</code>	enum	Canal de origen permitido

Fuente: `src/common/event_contract.py` y requisitos del proyecto integrador, sección 2.1.

La serialización JSON es determinista y la clave Kafka debe coincidir con `customer_id`. Una discrepancia, JSON inválido, UTF-8 inválido o campo fuera de contrato se deriva a `bank.transactions.invalid` con diagnóstico legible.

3.2 Tópicos, particiones y orden

Tópico	Part.	Clave	Política	Propósito
bank.transactions.raw	3	customer_id	retención 7 días	Input crudo y replay
risk.alerts.velocity	1	idempotency_key	compact,delete; 7 días	Revisiones de alertas
bank.transactions.invalid	1	event_id/hash	retención 7 días	Errores de contrato

Fuente: `compose.yaml`; `src/producer/skew_analysis.py` y `evidence/23-analisis-skew-offline.log`.

La entrada usa tres particiones para paralelizar clientes sin perder el orden por cliente. En una prueba de 300 eventos por escenario, la distribución balanceada fue 92/116/92; con una hot key al 80 % fue 258/20/22 y la mayor participación subió de 38,67 % a 86,00 %. Más particiones distribuyen clientes distintos, pero no dividen una misma clave. La salida usa una partición y compactación para facilitar la inspección y conservar la revisión vigente.

4. Pipeline Apache Beam

1. `KafkaIO` lee desde `bank.transactions.raw` con `auto.offset.reset=earliest`, `auto-commit` desactivado y `commit` en `finalize`.
2. `ParseAndValidateKafkaRecord` separa registros válidos e inválidos mediante una salida etiquetada.
3. `TimestampedValue` reemplaza el timestamp del elemento con `event_time` del payload.
4. `SlidingWindows` asigna ventanas de 60 segundos con período de 10 segundos.
5. `DeduplicateEvents` conserva `event_id` por cliente y ventana y programa limpieza con timer de watermark.
6. `CombinePerKey` usa `VelocityCombineFn` para conteo y suma incremental.
7. `CreateVelocityAlerts` construye una revisión solo cuando se supera al menos un umbral.
8. `WriteToKafka` publica alertas e inválidos con serialización determinista.

Modularidad: contrato, validación, política temporal, deduplicación, agregación, alertas y ejecución están separados en módulos y se prueban de forma independiente.

5. Tiempo de evento, ventanas y datos tardíos

Decisión	Configuración	Justificación
Tiempo	event_time del payload	La regla representa cuándo ocurrió la transacción.
Ventana	deslizante 60 s / paso 10 s	Reduce sensibilidad a cortes fijos y observa concentración.
Watermark	fuelle/runner	Determina cierre y temporización de panes.
Allowed lateness	30 s	Acepta correcciones acotadas y limita estado.
Triggers	EARLY 10 s; ON_TIME; LATE por elemento	Balancea latencia y completitud.
Acumulación	ACCUMULATING	Cada pane contiene el total vigente de la ventana.

Fuente: `src/pipeline/aggregation.py` y `tests/integration/test_pipeline_temporal.py`.

KafkaIO usa `ProcessingTime` como política de timestamp de fuente, pero la validación reasigna el timestamp desde el `event_time` validado antes del windowing. En la ejecución end-to-end acotada, el cierre de la fuente produjo panes `ON_TIME`. `TestStream` controla explícitamente el watermark y el desorden para demostrar un pane `ON_TIME` de tres eventos, un pane `LATE` acumulado de cuatro eventos, deduplicación y descarte posterior al horizonte de 30 segundos.

Modo de validación acotada: `--directrunner-smoke-mode` desactiva solo el disparo `EARLY` basado en tiempo de procesamiento; conserva tiempo de evento, ventanas, tolerancia a tardíos, acumulación, deduplicación y la política de revisiones.

6. Confiabilidad, idempotencia y semántica

Mecanismo	Garantía	Límite
Dedup por event_id	Primer evento por cliente y ventana	Horizonte <code>window_end + 30 s</code>
Clave estable de alerta	Mismo efecto lógico entre revisiones	El consumidor debe aplicar <code>upsert</code>
Productor Kafka idempotente	Reduce duplicados de reintento por sesión	No cubre todo el pipeline
Tópico compactado	Convergencia por clave	Compactación no es inmediata
Materializador SQLite	Conserva mayor revisión	Sin transacción Kafka-SQLite

Fuente: `src/pipeline/deduplication.py`, `src/pipeline/alerts.py` y `src/materializer/alert_consumer.py`.

La semántica declarada es at-least-once con efectos idempotentes. El consumidor hace `upsert` antes del `commit` del `offset`; un `replay` no modifica la vista si la revisión es duplicada u obsoleta. No se afirma `exactly-once end-to-end` porque no existe una transacción coordinada entre Kafka y SQLite ni una prueba de recuperación ante fallos.

7. Pruebas y evidencia

La suite contiene 71 casos. La verificación de referencia se ejecutó con Python 3.14.6 y el proyecto declara compatibilidad desde Python 3.12. Incluye contrato, validación, serialización, ventanas, agregación, deduplicación, alertas, productor, materializador y una prueba integral de tiempo con TestStream. Ruff valida calidad y formato.

Capa	Escenario	Resultado
Contrato	Campos inválidos y serialización	Errores acumulados; JSON determinista
Ventanas	Ventanas deslizantes 60/10	Seis ventanas por evento
Duplicados	event_id repetido	Una emisión por cliente/ventana
Temporal	ON_TIME, LATE y demasiado tardío	Corrección acumulada y cierre
End-to-end	8 eventos Kafka	3 alertas; 0 inválidos; LAG=0

Fuente: *tests/, evidence/01-verificacion-proyecto-final.log, evidence/22-resumen-validacion-final.md y evidence/23-analisis-skew-offline.log.*

7.1 Alertas observadas

Offset	Ventana UTC	Conteo	Total PYG	Condición
0	19:59:50-20:00:50	5	10.500.000	conteo + monto
1	19:59:40-20:00:40	4	8.000.000	conteo
2	20:00:00-20:01:00	4	8.000.000	conteo

Fuente: *evidence/21-alertas-directrunner-final-20260809-190216.log.*

Las tres salidas son ON_TIME, revision = 0 y ACCUMULATING. El orden de offsets no necesita coincidir con el orden cronológico de las ventanas porque cada clave se escribe como efecto independiente.

8. Operación reproducible

El README ofrece instalación, verificación, inicio de Kafka, ejecución del productor y del pipeline, consumo y detención. Compose contiene únicamente Kafka y la inicialización de tópicos; DirectRunner se ejecuta desde el entorno Python local y KafkaIO usa la imagen Java oficial de Beam.

1. Crear .venv e instalar el proyecto con dependencias de desarrollo.
2. Ejecutar scripts/verify.ps1.
3. Iniciar Kafka y kafka-init con Docker Compose.
4. Ejecutar scripts/run-pipeline-directrunner.ps1 en una terminal.
5. Publicar el escenario fijo desde una segunda terminal.
6. Consumir tres alertas mediante scripts/consume-alerts.ps1.

Reproducibilidad: la ejecución reproducible usa un base-time fijo y ocho eventos; un grupo único evita depender de offsets anteriores.

9. Límites y mejoras

1. DirectRunner está orientado a desarrollo y prueba, no a ejecución distribuida de producción.
2. La validación local usa un broker, factor de replicación 1 y una partición de salida.
3. La recuperación de estado y offsets no se prueba mediante una falla real con checkpoint.
4. SQLite demuestra persistencia local, no un sink distribuido ni una transacción exactly-once con Kafka.
5. Los umbrales son globales y no consideran segmentos o perfiles de riesgo.

El análisis reproducible de skew demuestra que una hot key permanece en una partición. Las mejoras restantes son un runner distribuido con checkpoints, métricas, monitoreo de dead-letter, mitigación con re-agregación y umbrales por segmento.

10. Contribución y trazabilidad con la rúbrica

El proyecto es individual. La autora realizó el diseño del caso PLD, contrato, arquitectura Kafka, pipeline Beam, política temporal, deduplicación, pruebas, validación end-to-end, documentación y presentación.

Referencias

1. Proyecto integrador: Data Streaming. Diseño e implementación de un pipeline end-to-end con Apache Kafka y Apache Beam. FPUNA, 2026.
2. Apache Beam. Programming Guide: windowing, triggers, watermarks and state.
<https://beam.apache.org/documentation/programming-guide/>
3. Apache Beam. TestStream.
https://beam.apache.org/releases/pydoc/current/apache_beam.testing.test_stream.html
4. Apache Beam. Kafka I/O connector.
https://beam.apache.org/releases/pydoc/current/apache_beam.io.kafka.html
5. Apache Kafka. Design documentation. <https://kafka.apache.org/documentation/#design>
6. Apache Kafka. Topic configuration reference.
<https://kafka.apache.org/documentation/#topicconfigs>
7. Confluent. librdkafka configuration properties.
https://docs.confluent.io/platform/current/clients/librdkafka/html/md_CONFIGURATION.html
8. Python Packaging User Guide. pyproject.toml specification.
<https://packaging.python.org/en/latest/specifications/pyproject-toml/>
9. Repositorio del proyecto: README, código fuente, pruebas y evidencias incluidas en la entrega.